38.2 在线追踪

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~260k tokens，输出 ~5k tokens（本节）

在多设备环境中，用户需要知道哪些设备正在连接、它们的状态如何。OpenClaw 的 **System Presence**（系统在线追踪）机制记录所有连接的设备信息，并实时同步到所有客户端。

38.2.1 Presence 数据结构

每个在线设备对应一条 `SystemPresence` 记录（`src/infra/system-presence.ts`）：

```
export type SystemPresence = {
  host?: string;              // 主机名
  ip?: string;                // IP 地址
  version?: string;           // OpenClaw 版本
  platform?: string;          // 平台信息（如 "macos 15.2"）
  deviceFamily?: string;      // 设备族（Mac / Linux / Windows）
  modelIdentifier?: string;   // 硬件型号（如 "Mac16,10"）
  lastInputSeconds?: number;  // 最后输入时间（秒前）
  mode?: string;              // 运行模式（gateway / node / cli）
  reason?: string;            // 连接原因（self / heartbeat / connect）
  deviceId?: string;          // 设备唯一标识
  roles?: string[];           // 角色列表（operator / node）
  scopes?: string[];          // 权限作用域
  instanceId?: string;        // 实例 UUID
  text: string;               // 人类可读的摘要文本
  ts: number;                 // 最后更新时间戳
};
```

自身 Presence 初始化

Gateway 启动时立即调用 `initSelfPresence()` 注册自身：

```
function initSelfPresence() {
  const host = os.hostname();
  const ip = resolvePrimaryIPv4();     // 优先 en0/eth0，回退其他接口
  const version = process.env.OPENCLAW_VERSION ?? "unknown";
  const modelIdentifier = (() => {
    if (os.platform() === "darwin") {
      return spawnSync("sysctl", ["-n", "hw.model"]).stdout.trim();
    }
    return os.arch();
  })();
  // 注册到 Presence Map
  entries.set(host.toLowerCase(), {
    host, ip, version, platform, deviceFamily, modelIdentifier,
    mode: "gateway", reason: "self",
    text: `Gateway: ${host} (${ip}) · app ${version} · mode gateway · reason self`,
    ts: Date.now(),
  });
}
```

这里用了同步的 `spawnSync("sysctl", ...)` ——因为 `initSelfPresence()` 在模块加载时执行（顶层调用），此时异步 API 会增加不必要的复杂性。而且 `sysctl` 的执行时间通常在 1 毫秒以内。

38.2.2 Presence 存储与过期

Presence 数据存储在模块级的 `Map<string, SystemPresence>` 中：

```
const entries = new Map<string, SystemPresence>();
const TTL_MS = 5 * 60 * 1000;  // 5 分钟过期
const MAX_ENTRIES = 200;         // 最多 200 条记录
```

`listSystemPresence()` 在返回数据前执行两项清理：

1. **TTL 过期清理**：删除超过 5 分钟未更新的记录
2. **容量限制**：如果记录数超过 200，按时间戳排序删除最旧的记录（LRU 策略）

```
export function listSystemPresence(): SystemPresence[] {
  ensureSelfPresence();
  const now = Date.now();
  // 清除过期记录
  for (const [k, v] of entries) {
    if (now - v.ts > TTL_MS) entries.delete(k);
  }
  // LRU 淘汰
  if (entries.size > MAX_ENTRIES) {
    const sorted = [...entries.entries()].toSorted((a, b) => a[1].ts - b[1].ts);
    const toDrop = entries.size - MAX_ENTRIES;
    for (let i = 0; i < toDrop; i++) entries.delete(sorted[i][0]);
  }
  touchSelfPresence();   // 刷新自身时间戳
  return [...entries.values()].toSorted((a, b) => b.ts - a.ts);
}
```

> **衍生解释**：LRU（Least Recently Used）是一种缓存淘汰策略。当缓存满时，优先淘汰最久未被访问的数据。这里的 Presence Map 本质上就是一个 LRU 缓存——超过容量时删除 `ts` 最小（最久未更新）的记录。

38.2.3 Presence 更新

当客户端（CLI、macOS 应用、iOS/Android 节点）连接或发送心跳时，Gateway 调用 `updateSystemPresence()` 更新 Presence：

```
export function updateSystemPresence(payload: SystemPresencePayload): SystemPresenceUpd
  ensureSelfPresence();
  const parsed = parsePresence(payload.text);
  // Key 选择优先级: deviceId > instanceId > host > ip > text 前缀
  const key = normalizePresenceKey(payload.deviceId)
    || normalizePresenceKey(payload.instanceId)
    || normalizePresenceKey(parsed.host)
    || parsed.ip
    || parsed.text.slice(0, 64);

  const existing = entries.get(key) ?? {};
  const merged = {
    ...existing,
    ...parsed,
    host: payload.host ?? parsed.host ?? existing.host,
    // ... 其他字段的合并逻辑
    roles: mergeStringList(existing.roles, payload.roles),
    ts: Date.now(),
  };
  entries.set(key, merged);

  // 返回变更摘要
  return { key, previous: existing, next: merged, changes, changedKeys };
}
```

合并策略遵循"新值优先"原则： `payload` （最新的客户端数据） > `parsed` （从 text 解析的数据） > `existing` （已有记录）。角色和作用域列表使用集合合并（ `mergeStringList` ），避免重复项。

文本解析

客户端发送的 Presence 数据包含一个人类可读的 `text` 字段，格式为：

```
Node: myhost (192.168.1.10) · app 1.2.3 · last input 42s ago · mode node · reason heart
```

`parsePresence()` 使用正则表达式从这个文本中提取结构化字段，这是一种兼容性设计——即使客户端只发送 `text` 字段而没有结构化数据，Gateway 也能解析出关键信息。

38.2.4 Presence 版本与广播

Presence 变更会触发版本号递增和实时广播：

```
let presenceVersion = 1;

export function incrementPresenceVersion(): number {
  presenceVersion += 1;
  return presenceVersion;
}
```

Gateway 在以下时机广播 Presence 更新：

1. **新客户端连接**时
2. **客户端断开**时
3. **心跳更新**时
4. **设备配对/取消配对**时

客户端（如 Web 控制台）收到 Presence 更新后，可以实时显示在线设备列表、打字指示器等 UI 元素。

38.2.5 打字指示器

> **衍生解释**：打字指示器（Typing Indicator）是即时通讯中常见的功能——当对方正在输入消息时，显示"正在输入..."的提示。在 OpenClaw 中，打字指示器的含义更广泛：它不仅包括用户正在输入，还包括 Agent 正在"思考"（调用 LLM API）和正在输出流式响应。

OpenClaw 的打字指示器通过 Presence 系统实现。当 Agent 开始处理用户消息时，Gateway 会向所有客户端广播 `mode` 变更：

```
mode: "idle"     → 空闲
mode: "thinking" → AI 正在思考
mode: "writing"  → 正在流式输出
mode: "tool"     → 正在执行工具调用
```

Web 控制台和原生客户端监听这些 mode 变化来显示相应的 UI 状态（如三点动画、进度指示器等）。

本节小结

1. **SystemPresence** 记录每个连接设备的详细信息：主机名、IP、平台、硬件型号、运行模式、版本等。Gateway 启动时自动注册自身。

2. **TTL + LRU** 双重机制控制 Presence 数据的生命周期：5 分钟过期自动清除，超过 200 条时淘汰最旧记录。
3. **Presence 更新**采用"新值优先"的合并策略，Key 选择支持 deviceId、instanceId、host 多级回退，确保同一设备的记录不会重复。
4. **版本号广播**让客户端能以最小开销感知 Presence 变更——只需比较一个整数即可判断是否需要拉取最新数据。
5. **打字指示器**基于 Presence 的 `mode` 字段实现，覆盖了 idle/thinking/writing/tool 四种状态。